

UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computação
Ciência da Computação

Disciplina: Inteligência Artificial

Relatório Técnico

Lista 2 de Exercícios de IA - AB2

Período 2026.1

Professor

Evandro de Barros Costa

Integrantes

Hugo Stankowich Souza

Lucca Paes Costa

Renato Coca Terrazas Freire

Samuel Medino da Silva

Caio Daniel de Jesus Cavalcante

Maceió - AL

2026

Contents

Relatório Técnico - Lista 2 de Exercícios de IA - AB2 2026.1	1
1. Introdução	1
2. Questão 1 - Shell Genérica para Sistemas Baseados em Conhecimento	1
2.1 Objetivo e Visão Geral	1
2.2 Arquitetura Implementada	2
2.3 Representação do Conhecimento	2
2.4 Editor da Base de Conhecimento	2
2.5 Motor de Inferência	2
2.6 Mecanismo de Explicação	2
2.7 Interface e Exemplos de Consulta	3
3. Questão 2.1 - Sistema Akinator no Domínio Animais	3
4. Questão 2.4 - Diagnóstico Médico Baseado em Casos	4
5. Questão 3 - Aplicação com Agente Baseado em LLM	5
6. Conclusão	6
7. Referências e Artefatos	6

Relatório Técnico - Lista 2 de Exercícios de IA - AB2 2026.1

1. Introdução

Este relatório consolida as implementações desenvolvidas para a Lista 2 de Exercícios de IA, AB2, da disciplina Inteligência Artificial. O documento foi estruturado de acordo com os entregáveis solicitados no enunciado: uma ferramenta genérica para sistemas baseados em conhecimento, duas aplicações entre as opções da Questão 2 e uma aplicação envolvendo agentes baseados em LLM.

As implementações entregues foram: (i) uma shell genérica de sistema especialista com encadeamento para frente, para trás e híbrido; (ii) um Akinator no domínio de animais; (iii) um sistema de diagnóstico médico baseado em casos; e (iv) um agente conversacional com LLM que utiliza o motor CBR como ferramenta externa.

Questão	Implementação entregue	Atendimento ao enunciado
1	Expert Shell - sistema especialista genérico	Shell reutilizável, editor de base, regras SE...ENTÃO, inferência FC/BC/híbrida, explicação e aplicação demonstrativa médica.
2.1	Akinator - domínio animais	22 entidades, 20 atributos, respostas Sim/Não/Não sei e eliminação progressiva de hipóteses.
2.4	CBR Médico	17 casos fictícios, 7 doenças, similaridade por cosseno e ciclo Retrieve, Reuse, Revise e Retain.
3	Agente LLM + CBR	Agente baseado em Claude com tool use, memória multi-turno e consulta ao CBR.

2. Questão 1 - Shell Genérica para Sistemas Baseados em Conhecimento

2.1 Objetivo e Visão Geral

A Questão 1 solicitou uma ferramenta genérica, nos moldes do Expert SINTA, para construção de aplicações de diagnóstico e recomendação baseadas em fatos e regras. A implementação denominada Expert Shell atende a esse objetivo ao separar o motor de inferência do domínio de aplicação. Assim, um especialista pode alterar ou criar uma base de conhecimento em JSON sem modificar o código-fonte.

Como domínio demonstrativo foi construída uma base de diagnóstico médico com 28 regras de produção, 36 fatos possíveis e 11 hipóteses de diagnóstico, superando os mínimos exigidos pelo enunciado.

Item exigido	Implementação realizada
Pelo menos 20 regras	28 regras no arquivo expert_shell/domains/medical.json.
Pelo menos 30 fatos possíveis	36 atributos em possible_values.
Pelo menos 5 hipóteses	11 hipóteses diagnósticas.
Persistência da base	Leitura e gravação em JSON via KnowledgeBase.save/load.
Explicação	Registro de metas, perguntas, regras disparadas e trilha de inferência.

2.2 Arquitetura Implementada

A arquitetura foi organizada em módulos independentes: base de conhecimento, editor, motores de inferência, mecanismo de explicação, interface de linha de comando e ponte opcional com LLM. Essa separação reforça a reutilização da shell em diferentes domínios.

```
expert_shell/  
  knowledge_base/  kb.py, editor.py  
  inference/       forward.py, backward.py, hybrid.py  
  explanation/     explainer.py  
  interface/       cli.py  
  llm/             bridge.py  
  domains/         medical.json  
  main.py
```

2.3 Representação do Conhecimento

O conhecimento é representado por fatos no formato atributo = valor, regras de produção e hipóteses. As regras seguem exatamente o padrão pedido no enunciado: SE condição1 E condição2 E ... ENTÃO conclusão. Cada regra possui identificador, nome, condições, conclusão e descrição textual.

```
Exemplo conceitual:  
SE febre = alta E tosse = seca E dor_muscular = sim E fadiga = sim  
ENTÃO suspeita = gripe
```

A base demonstrativa cobre diagnósticos como gripe, COVID-19, dengue, dengue hemorrágica, pneumonia, diabetes tipo 2, hipertensão arterial, alergia respiratória, gastroenterite, infecção urinária e anemia.

2.4 Editor da Base de Conhecimento

O módulo KBEditor implementa cadastro, listagem, alteração e remoção de fatos, regras, hipóteses e valores possíveis. A persistência em JSON permite que a base seja carregada, editada e salva sem alterar o motor de inferência.

2.5 Motor de Inferência

Foram implementadas as três estratégias obrigatórias. O encadeamento para frente parte dos fatos conhecidos e dispara todas as regras aplicáveis até não haver novas conclusões. O encadeamento para trás parte de uma hipótese e tenta prová-la recursivamente, solicitando ao usuário apenas as informações necessárias. A estratégia híbrida combina as duas: primeiro propaga fatos conhecidos, depois avalia hipóteses com backward chaining e, após novas respostas, executa forward chaining para propagar conclusões intermediárias.

Estratégia	Papel no sistema
Forward Chaining	Dispara regras cujas condições já estão satisfeitas e registra fatos inferidos.
Backward Chaining	Tenta provar objetivos como suspeita = dengue, perguntando fatos faltantes ao usuário.
Híbrida	Executa FC, usa BC por hipótese e reexecuta FC para propagar cada novo fato.

2.6 Mecanismo de Explicação

O mecanismo de explicação responde às perguntas Por quê? e Como?, conforme solicitado. Para Por quê?, o sistema registra a regra ou hipótese que motivou uma pergunta. Para Como?, o sistema lista as regras ativadas, as condições satisfeitas e a conclusão obtida.

Por quê? -> Perguntei sobre exposicao_aedes porque é condição da regra R09, que tenta provar suspeita = dengue.
 Como? -> A regra R09 foi ativada a partir de exposicao_aedes = sim, febre = alta, enjoo = sim e dor_cabeca = sim.

2.7 Interface e Exemplos de Consulta

A interface é em linha de comando. O usuário pode iniciar uma consulta, escolher estratégia de inferência, editar a base, carregar ou salvar JSON e consultar a trilha de inferência. Durante a consulta, os comandos por que, como e trilha ativam as explicações.

Diagnóstico esperado	Fatos informados
Gripe	febre = alta, tosse = seca, dor_muscular = sim, fadiga = sim
COVID-19	perda_olfato = sim, perda_paladar = sim
Dengue	exposicao_aedes = sim, febre = alta, enjoo = sim, dor_cabeca = sim
Pneumonia	tosse = produtiva, falta_ar_repouso = sim, febre = alta, exame_rx_torace = infiltrado

2.8 Limitações e Melhorias

- A base médica é educacional e não deve ser usada como diagnóstico real.
- O mecanismo é determinístico; incerteza clínica poderia ser modelada por pesos ou redes probabilísticas.
- A interface CLI é funcional, mas uma interface web facilitaria a demonstração e o uso por especialistas.

3. Questão 2.1 - Sistema Akinator no Domínio Animais

Entre as quatro opções da Questão 2, uma das aplicações escolhidas foi o sistema de perguntas e respostas no estilo Akinator. O domínio selecionado foi Animais, com 22 entidades e 20 atributos booleanos. O sistema formula perguntas sequenciais e elimina hipóteses incompatíveis com as respostas.

Requisito	Atendimento
Pelo menos 20 entidades	22 animais cadastrados.
Pelo menos 15 atributos	20 atributos/características.
Sim, Não ou Não sei	Entradas s, n e ns implementadas na interface.
Hipótese mais provável	Quando há múltiplos candidatos, o sistema apresenta o melhor palpite e os candidatos restantes.

3.1 Representação e Inferência

A base é um JSON com dois blocos: attributes, contendo as perguntas, e entities, contendo os animais e seus vetores booleanos de atributos. A inferência usa busca em espaço de hipóteses com eliminação progressiva. A cada rodada, o sistema escolhe o atributo que melhor balanceia os candidatos restantes, filtra as entidades de

acordo com Sim ou Não e mantém todos os candidatos quando o usuário responde Não sei.

Essa estratégia é semelhante a construir dinamicamente uma árvore de decisão: perguntas que dividem melhor o conjunto de candidatos reduzem a quantidade esperada de interações.

3.2 Exemplos e Resultados

Caso	Sequência resumida	Resultado
Tigre	mamífero, grande porte, carnívoro, não aquático, não africano, asiático, ameaçado	Identificado em 7 perguntas.
Abelha	não mamífero, não ave, não réptil, não peixe, inseto, voa	Identificada em 6 perguntas.
Pinguim Imperador	não mamífero, ave, não voa	Identificado em 3 perguntas.

Nos testes internos relatados, a média observada ficou em torno de 7 perguntas por sessão. A taxa de acerto foi de 100% sem respostas Não sei e aproximadamente 90% quando respostas desconhecidas foram usadas. As falhas ocorrem quando muitas características distintivas são ignoradas, por exemplo Leão vs Lobo sem origem geográfica ou Águia Careca vs Papagaio sem atributos alimentares/domésticos.

3.3 Limitações e Melhorias

- O sistema ainda não aprende automaticamente novos animais quando erra.
- A representação booleana não captura atributos graduais ou incerteza.
- Uma expansão da base exigiria técnicas mais robustas de seleção de perguntas e persistência de aprendizado.

4. Questão 2.4 - Diagnóstico Médico Baseado em Casos

A segunda aplicação escolhida da Questão 2 foi o sistema de Raciocínio Baseado em Casos (CBR) para apoio educacional ao diagnóstico médico. O sistema recebe sintomas, compara o novo caso com uma base de casos fictícios e sugere diagnóstico e tratamento a partir dos casos mais similares.

Requisito	Atendimento
Pelo menos 15 casos	17 casos médicos fictícios.
No mínimo 5 doenças	7 diagnósticos: gripe, COVID-19, dengue, pneumonia bacteriana, malária, resfriado comum e gastroenterite viral.
Similaridade entre casos	Similaridade por cosseno sobre vetores binários de 20 sintomas.
Exibir casos semelhantes	Retorna top-3 casos mais similares.
Incluir novos casos	Etapa Retain salva novos casos validados em cases.json.

4.1 Base e Representação dos Casos

Cada caso é armazenado em JSON com identificador, vetor binário de sintomas, diagnóstico, tratamento e marcação de validação. Os 20 sintomas considerados incluem febre, tosse, dor de cabeça, fadiga, dor de garganta, coriza, dificuldade respiratória, dor muscular, calafrios, náusea, vômito, diarreia, dor abdominal, perda de olfato, manchas na pele, dor articular, irritação ocular, falta de apetite, sudorese noturna e confusão mental.

4.2 Método de Similaridade

A similaridade por cosseno mede a co-ocorrência de sintomas presentes entre o caso consultado e cada caso armazenado. Ela é adequada para vetores binários porque evita que dois casos pareçam semelhantes apenas por compartilharem muitos sintomas ausentes.

$$\text{sim}(A,B) = \text{soma}(A_i * B_i) / (\text{sqrt}(\text{soma}(A_i^2)) * \text{sqrt}(\text{soma}(B_i^2)))$$

4.3 Ciclo CBR Implementado

Etapa	Implementação
Retrieve	Calcula similaridade entre a consulta e todos os casos, retornando top-3.
Reuse	Usa o diagnóstico e tratamento do caso mais similar como solução candidata.
Revise	Permite ao usuário confirmar ou corrigir o diagnóstico sugerido.
Retain	Salva o caso validado/corrigido com novo ID automático.

4.4 Exemplos e Análise

Consulta	Casos recuperados	Interpretação
Dengue	C005 94,3%; C006 71,2%; C010 68,7%.	Alta confiança para dengue; malária aparece como diferencial por sintomas sobrepostos.
COVID-19	C003 91,7%; C004 88,4%; C001 52,1%.	Perda de olfato e dificuldade respiratória diferenciam COVID-19 de gripe.
Resfriado comum	C011 100,0%; C017 81,6%; C012 57,7%.	Sintomas leves e respiratórios superiores levam a resfriado.

A taxa de acerto interna relatada foi de aproximadamente 88% no top-1. As principais limitações são doenças com sintomas muito parecidos, ausência de pesos por sintoma, falta de variáveis contínuas e base pequena para um contexto clínico real.

5. Questão 3 - Aplicação com Agente Baseado em LLM

A Questão 3 solicitou uma aplicação envolvendo agentes baseados em LLM. Foi desenvolvido um agente de triagem médica que combina linguagem natural com o motor CBR da Questão 2.4. O LLM atua como orquestrador: interpreta a mensagem do usuário, decide quais ferramentas chamar, consulta a base CBR e sintetiza a resposta final.

5.1 Arquitetura do Agente

```

Usuário em linguagem natural
-> main.py mantém conversa multi-turno
-> agent.py chama Claude com ferramentas disponíveis
-> list_symptoms informa o vocabulário de sintomas
-> search_medical_cases consulta o CBR por similaridade
-> Claude interpreta os resultados e responde ao usuário

```

O agente usa o SDK da Anthropic e define ferramentas com JSON Schema. Quando o modelo retorna uma solicitação de tool_use, o código executa a função Python correspondente e devolve o resultado ao modelo como observação. Esse ciclo se repete até a resposta final.

Componente	Função
main.py	Interface CLI e histórico multi-turno.
agent.py	Loop do agente, chamada ao modelo, detecção de tool_use e retorno da resposta final.
tools.py	Implementa list_symptoms e search_medical_cases.
../cbr_medico	Fornece a base de casos e o motor de similaridade.

5.2 Ferramentas e Funcionamento

- list_symptoms retorna os 20 sintomas rastreados pela base CBR.
- search_medical_cases recebe sintomas binários, consulta o motor CBR e retorna os 3 casos mais similares com diagnóstico e tratamento.
- O LLM não substitui o raciocínio simbólico/case-based: ele apenas interpreta linguagem natural, chama ferramentas e explica resultados.

5.3 Exemplo de Interação

Usuário: Estou com febre alta, perdi o olfato e tenho dificuldade para respirar.
[ferramenta: list_symptoms]
[ferramenta: search_medical_cases]
Agente: O sistema encontrou maior similaridade com casos de COVID-19,
apresenta confiança, tratamento sugerido e diagnósticos diferenciais.
Ao final, informa que a aplicação é educacional.

5.4 Limitações

- Depende de chave de API Anthropic e conectividade externa.
- A qualidade da resposta textual depende do mapeamento correto dos sintomas para o vocabulário do CBR.
- A base médica é fictícia e educacional, portanto não deve orientar decisões clínicas reais.

6. Conclusão

O conjunto de implementações atende às solicitações da Lista 2. A Questão 1 foi coberta por uma shell genérica de sistema especialista, com editor, base persistente, inferência para frente, para trás e híbrida, mecanismo de explicação e domínio demonstrativo com tamanho superior ao mínimo exigido. A Questão 2 foi atendida por duas aplicações: um Akinator baseado em eliminação de hipóteses e um sistema CBR médico completo com ciclo 4R. A Questão 3 foi atendida por um agente LLM que integra linguagem natural e chamadas de ferramenta sobre o motor CBR.

Em termos de aprendizagem de IA, o trabalho demonstra diferentes formas de representação e raciocínio: regras de produção, busca em espaço de hipóteses, raciocínio baseado em casos e agentes com LLM. Também evidencia as diferenças entre raciocínio simbólico determinístico, similaridade vetorial e orquestração por modelo de linguagem.

7. Referências e Artefatos

- Enunciado: Lista 2 de Exercícios de IA, AB2, Prof. Evandro Costa, Inteligência Artificial, 2026.1.
- Código-fonte: diretórios expert_shell, akinator, cbr_medico e agente_llm.
- Bases de conhecimento: expert_shell/domains/medical.json, akinator/knowledge_base.json e cbr_medico/cases.json.
- Documentação técnica local: arquivos RELATORIO_TECNICO.md de cada implementação.